

Relatório de Análise Experimental: Tabela Hash com Encadeamento Separado

Disciplina: Resolução de Problemas Estruturados

Aluno: Emmanuel Antonnietti Ribeiro dos Santos; Juliano Cesar Enns Miranda Marcos

Data: 27/11/2025

1. Introdução

O objetivo deste experimento foi implementar e analisar o desempenho de uma Tabela Hash utilizando a estratégia de tratamento de colisões por **Encadeamento Separado**. O foco principal do estudo foi comparar a eficiência de três diferentes funções de hash — Divisão, Multiplicação e Dobramento — sob diferentes fatores de carga e tamanhos de tabela.

A implementação foi realizada em linguagem Java ("Java Básico"), sem o uso de bibliotecas de coleções (como HashMap ou ArrayList), garantindo o controle manual sobre a alocação de nós e o gerenciamento das listas encadeadas.

2. Metodologia

O experimento consistiu na execução de testes automatizados variando três parâmetros principais:

1. **Tamanho da Tabela (\$M\$):** 1009, 10007 e 100003 (números primos para favorecer a dispersão).
2. **Tamanho do Dataset (\$N\$):** 1.000, 10.000 e 100.000 chaves inteiras de 9 dígitos.
3. **Funções de Hash (\$h(k)\$):**

Divisão (\$H_DIV\$): $h(k) = k \bmod M$.

Multiplicação (\$H_MUL\$): Utilizando a constante áurea inversa $A \approx 0.618$.

Dobramento (\$H_FOLD\$): Soma de blocos de 3 dígitos da chave.

Para garantir a reprodutibilidade, foram utilizadas três sementes fixas para a geração pseudoaleatória dos dados. As métricas coletadas incluíram o tempo de execução (em milissegundos), o número de colisões na tabela (compartimentos ocupados) e o número de colisões na lista (nós percorridos).

Conforme preconizado na teoria de hashing, a eficiência da tabela depende diretamente da capacidade da função de espalhar as chaves uniformemente. **Distribuições mais uniformes reduzem o custo médio no encadeamento separado.**

3. Resultados e Análise

3.1. Análise de Inserção e Fator de Carga

Observou-se que o tempo de inserção cresce linearmente em relação ao tamanho do dataset (N), o que é esperado para inserções em tabelas hash ($O(1)$ médio). No entanto, houve uma discrepância significativa entre as funções avaliadas.

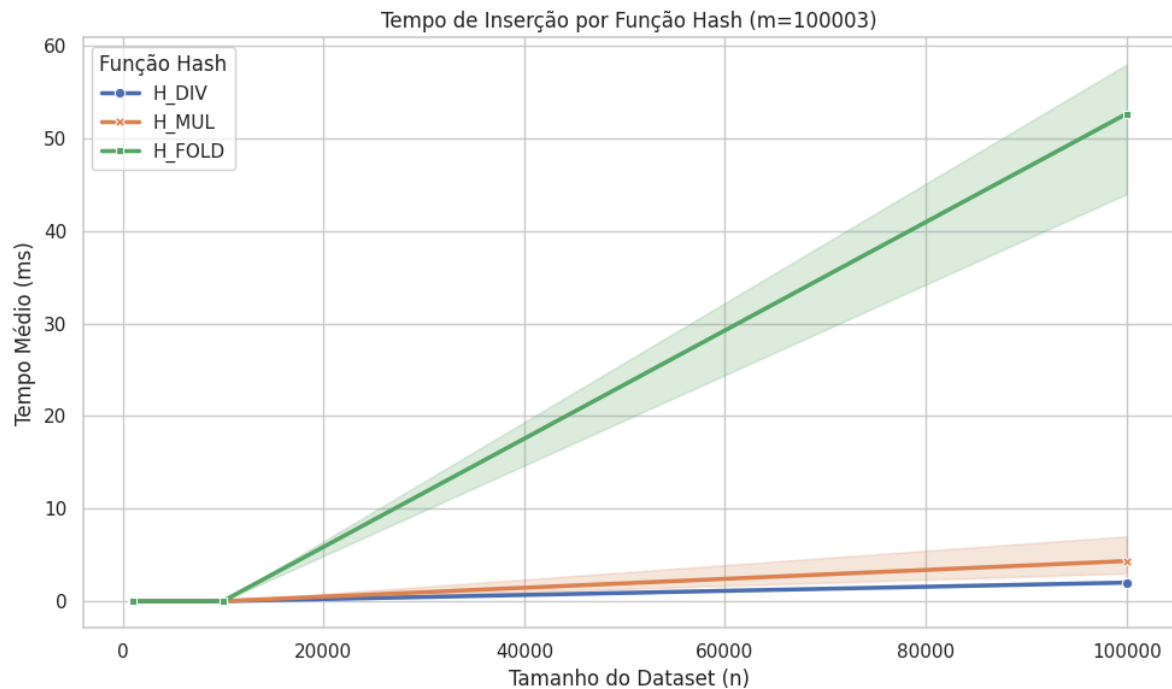


Figura 1: Tempo médio de inserção para $M=100.003$.

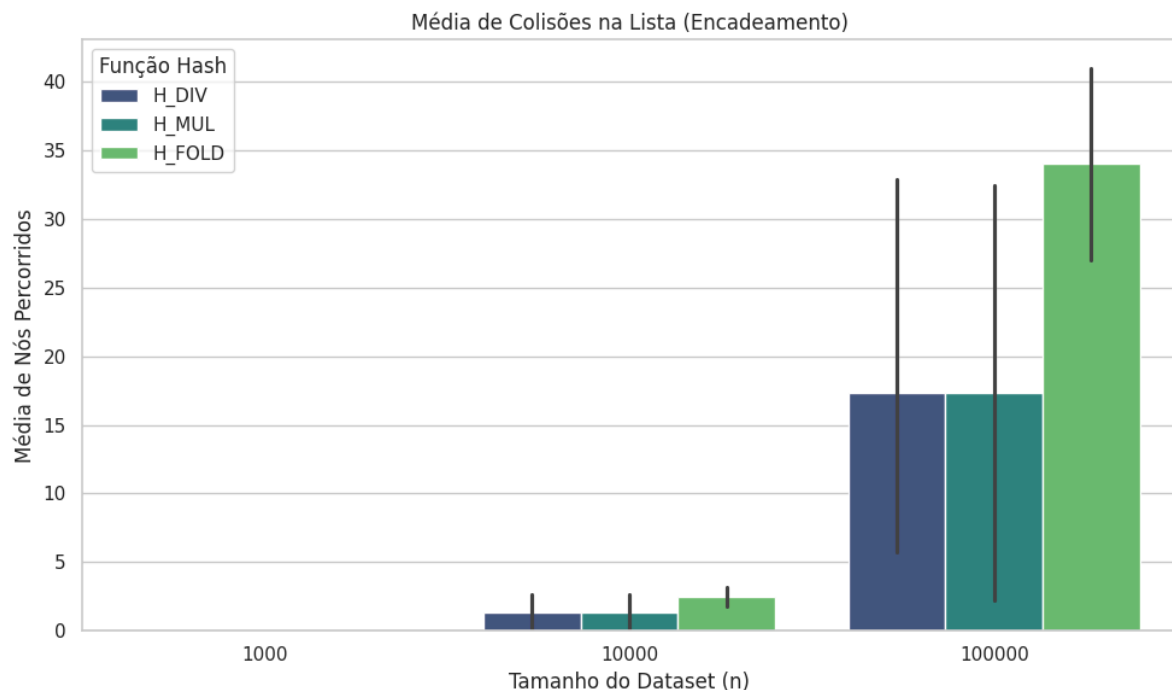


Figura 2: Média de colisões na lista (comprimento médio das cadeias).

A função de **Dobramento (H_FOLD)** apresentou o pior desempenho tanto em tempo quanto em número de colisões na lista (Figura 2). Enquanto as funções de Divisão e Multiplicação mantiveram as listas encadeadas curtas, o Dobramento gerou listas significativamente mais longas (média superior a 30 nós para $N=100.000$).

Isso indica que o método de somar blocos de 3 dígitos gerou muitos valores de hash idênticos após a aplicação do módulo, criando aglomerados (*clusters*) em poucos índices da tabela. Esse fenômeno aumentou o custo da inserção, pois o algoritmo precisa percorrer a lista encadeada até o último nó para inserir o novo elemento.

3.2. Análise de Busca: Sucesso (Hit) vs. Fracasso (Miss)

A eficiência da busca no encadeamento separado está intimamente ligada ao comprimento médio das listas ($\alpha = N/M$).

- **Busca com Sucesso:** Em média, percorre-se metade da lista onde o elemento se encontra.
- **Busca com Fracasso:** É necessário percorrer a lista inteira para confirmar a ausência do elemento.

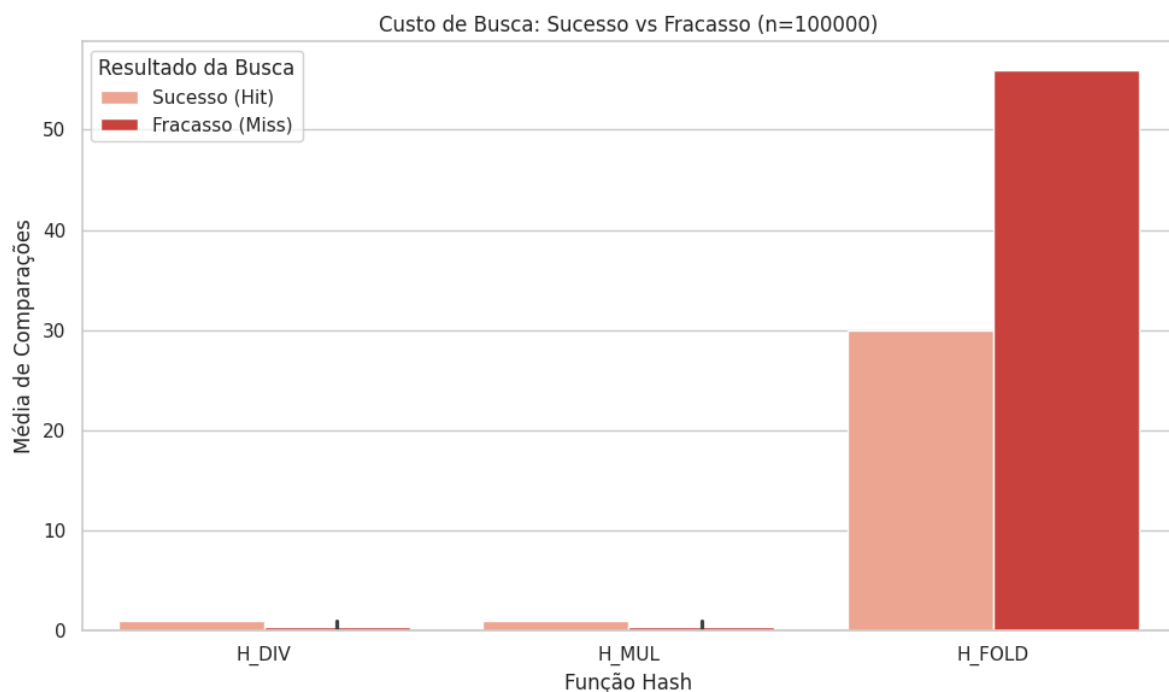


Figura 3: Comparação de custo entre buscas bem-sucedidas e malsucedidas.

Conforme ilustrado na Figura 3, o custo computacional para o **Fracasso (Miss)** é sistematicamente maior ou igual ao custo do **Sucesso (Hit)**.

No caso da função H_FOLD, a disparidade foi acentuada. Devido à má distribuição das chaves e à formação de longas listas encadeadas, a busca por elementos inexistentes tornou-se custosa, exigindo dezenas de comparações de chaves. Já para H_DIV e H_MUL,

que distribuíram as chaves de forma quase uniforme, o número de comparações manteve-se baixo (próximo a 1 ou 2), aproximando-se do ideal teórico de $O(1)$.

4. Conclusão

O experimento demonstrou que a escolha da função de hash é crítica para o desempenho de tabelas com encadeamento separado. Embora a estrutura suporte fatores de carga altos ($\alpha \approx 1$ ou até maiores), o desempenho degrada rapidamente se a função hash não garantir uniformidade.

As funções de **Divisão** e **Multiplicação** mostraram-se robustas e eficientes para chaves inteiras aleatórias, mantendo o custo das operações baixo. Em contrapartida, a função de **Dobramento** implementada mostrou-se inadequada para este domínio de dados específico, resultando em alto número de colisões e degradação do tempo de resposta, especialmente em operações de busca malsucedida.